📊 Credit Scoring Analytics – Banking Credit Risk Analysis

Fecha de Creación : Octubre 2022

- **Notebook** by Julio César Martínez I.
- **Supported** by Francisco Alfaro & Alfonso Tobar
- **Code Reviewer** Oscar Flores

Licencia

Copyright @2023 by Julio César Martínez Izaguirre

Licensed under the Apache License, Version 2.0 (the "License"); you may not use this file except in compliance with the License. You may obtain a copy of the License at

http://www.apache.org/licenses/LICENSE-2.0

Unless required by applicable law or agreed to in writing, software distributed under the License is distributed on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License

Tabla de Contenido

---

1. Propósito del Proyecto
2. Etapa 1 : Carga de Datos
3. Etapa 2 : Transformación de Datos
4. Etapa 3 : Trabajando valores ausentes
5. Etapa 4 : Clasificación de datos
6. Etapa 5 : Comprobación de hipótesis
7. Conclusión General

Introducción

Este proyecto consiste en **preparar un informe** para la división de préstamos de un banco. Deberemos averiguar si el estado civil y el número de hijos de un cliente tienen un impacto en el incumplimiento de pago de un préstamo. El banco ya tiene algunos datos sobre la solvencia crediticia de los clientes.

Nuestro informe se tendrá en cuenta al crear una **puntuación de crédito** para un cliente potencial. La **puntuación de crédito** se utiliza para evaluar la capacidad de un prestatario potencial para pagar su préstamo.

Propósito del Proyecto

Dar a conocer si un cliente puede ser acreedor o no a una línea de crédito por parte del banco de acuerdo a una serie de características personales y profesionales registradas por la entidad financiera.

Hipótesis

1. Averiguar si existe alguna conexión entre tener hijos y pagar un prestamo a tiempo.
2. Determinar si existe una conexión entre el estado civil y el pago a tiempo de un préstamo.
3. Saber si existe alguna conexión con el nivel de ingresos y el pago de un préstamo.
4. Cómo afectan los diferentes propósitos del préstamo al reembolso a tiempo del préstamo.

Descripción de los datos

Estas son las columnas que hay en nuestro dataset y una breve descripción de ellas.

- `children` - el número de hijos en la familia
- `days_employed` - experiencia laboral en días
- `dob_years` - la edad del cliente en años
- `education` - la educación del cliente
- `education_id` - identificador de educación
- `family_status` - estado civil
- `family_status_id` - identificador de estado civil

- `gender` - género del cliente
- `income_type` - tipo de empleo
- `debt` - ¿había alguna deuda en el pago de un préstamo?
- `total_income` - ingreso mensual
- `purpose` - el propósito de obtener un préstamo

Etapa 1 : Carga de Archivos y Exploración de Datos

Vamos a comenzar con lo básico, vamos a cargar las librerías que vamos a utilizar para este proyecto y enseguida haremos una breve exploración de datos inicial para conocer lo que hay en nuestro set.

Carga de Librerias

In [1]:
```
import pandas as pd
import numpy as np

pd.options.display.max_rows = 100
```

Lectura de Datos

In [ ]:
```
dataclients = pd.read_csv('/content/credit_scoring_eng.csv')
```

Exploración de Datos

In [ ]:
```
dataclients.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 21525 entries, 0 to 21524
Data columns (total 12 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   children          21525 non-null  int64
 1   days_employed     19351 non-null  float64
 2   dob_years         21525 non-null  int64
 3   education         21525 non-null  object
 4   education_id      21525 non-null  int64
 5   family_status     21525 non-null  object
 6   family_status_id  21525 non-null  int64
 7   gender            21525 non-null  object
 8   income_type       21525 non-null  object
 9   debt              21525 non-null  int64
 10  total_income      19351 non-null  float64
 11  purpose           21525 non-null  object
dtypes: float64(2), int64(5), object(5)
memory usage: 2.0+ MB
```

In [ ]:
```
dataclients.head()
```

Out[ ]:

	children	days_employed	dob_years	education	education_id	family_status
0	1	-8437.673028	42	bachelor's degree	0	married
1	1	-4024.803754	36	secondary education	1	married
2	0	-5623.422610	33	Secondary Education	1	married
3	3	-4124.747207	32	secondary education	1	married
4	0	340266.072047	53	secondary education	1	civil partnership

> Podemos ver que existen valores ausentes `NaN` en las columnas `days_employed` y `total_income`. Además, encontramos algunos valores negativos en columnas que vamos a investigar más adelante, por lo pronto analicemos un poco más los valores ausentes que hemos encontrado.

Examinando valores ausentes

In [ ]:
```
dataclients[dataclients.isna().any(axis=1)]
```

Out[ ]:

	children	days_employed	dob_years	education	education_id	family_
12	0	NaN	65	secondary education	1	part
26	0	NaN	41	secondary education	1	
29	0	NaN	63	secondary education	1	un
41	0	NaN	50	secondary education	1	
55	0	NaN	54	secondary education	1	part
...	...	...	...	...	...	
21489	2	NaN	47	Secondary Education	1	
21495	1	NaN	50	secondary education	1	part
21497	0	NaN	48	BACHELOR'S DEGREE	0	
21502	1	NaN	42	secondary education	1	
21510	2	NaN	28	secondary education	1	

2174 rows × 12 columns

> Los **valores ausentes** parecen estar relacionados entre ambas columnas, es decir, son simétricos. Para estar seguros de esto, vamos a realizar un recuento de los valores ausentes en ambas columnas.

In [ ]:
```
dataclients[['days_employed', 'total_income']].isna().sum()
```

Out[ ]:
```
days_employed    2174
total_income     2174
dtype: int64
```

> El número de filas de la tabla filtrada coincide con el número de valores ausentes, esto quiere decir que las personas que no tienen experiencia laboral tampoco registran un ingreso mensual.

Supuesto

Lo anterior probablemente tenga una relación con el nivel de educación ya que una educación deficiente o de bajo nivel tiene como consecuencias desempleo y/o empleos no formales, para ello, vamos a investigar a los clientes que no tienen datos sobre la característica identificada y la columna con los valores ausentes.

Análisis de Educación

In [ ]:
```
dataclients['education'].unique()
```

Out[ ]:
```
array(["bachelor's degree", 'secondary education', 'Secondary Educati
on',
       'SECONDARY EDUCATION', "BACHELOR'S DEGREE", 'some college',
       'primary education', "Bachelor's Degree", 'SOME COLLEGE',
       'Some College', 'PRIMARY EDUCATION', 'Primary Education',
       'Graduate Degree', 'GRADUATE DEGREE', 'graduate degree'],
      dtype=object)
```

> Podemos ver datos duplicados que contienen el mismo grado escolar pero escritos en mayúsculas y minúsculas, vamos arreglar un poco esto para tener un resultado más preciso.

In [ ]:
```
dataclients['education'] = dataclients['education'].str.lower()
dataclients['education'].unique()
```

Out[ ]:
```
array(["bachelor's degree", 'secondary education', 'some college',
       'primary education', 'graduate degree'], dtype=object)
```

Filtrando Tabla

Una vez limpios nuestros datos haremos un filtro para conocer los valores ausentes en el nivel de educación secundaria

In [ ]:
```
dataclients[dataclients['education'] == 'secondary education']['days_e
```

Out[ ]:
```
 NaN              1540
-4024.803754         1
 400147.947311       1
-3507.818775         1
-635.915050          1
                   ...
-5730.178239         1
-429.448441          1
-1613.140030         1
-452.930653          1
-1984.507589         1
Name: days_employed, Length: 13694, dtype: int64
```

> Nuestros datos nos indican que existen un total de **13,694** registros de nivel de **educación secundaria**, de los cuales hay un total de 1,540 datos sobre los valores ausentes, ahora, comprobemos la distribución entre los datos ausentes de nivel secundaria y el total de clientes.

In [ ]:
```
nan_data = 2174
nan_sec_edu = 1540

total_dis = nan_sec_edu / nan_data
f'El porcentaje de distribución del bajo nivel educativo es de: {(tota
```

Out[ ]:
```
'El porcentaje de distribución del bajo nivel educativo es de: 71%'
```

Posibles razones por las que hay valores ausentes en los datos

El resultado nos muestra que un 71% de los valores ausentes proviene de clientes que tienen un bajo nivel educativo. Esto se acerca más a nuestra teoría de que sean clientes desempleados y/o tengan empleos informales.

Sin embargo esto no cubre el 10% de los valores ausentes por lo cuál no podemos definir si se trata de un patrón exacto o son valores aleatorios. Vamos a comprobar si nuestros valores ausentes son al azar o hay un patrón con esto.

In [ ]:
```
# Comprobando la distribución en el conjunto de datos entero
totaldata = len(dataclients)
nan_data = 2174

distribution = nan_data / totaldata
f'La distribución del conjunto de datos entero es: {(distribution):.0%
```

Out[ ]:
```
'La distribución del conjunto de datos entero es: 10%'
```

> Podemos ver que la distribución de ambos conjuntos es muy diferente, esto significa que no se cubre el 100% de los valores ausentes en nuestra tabla filtrada lo cual quiere decir que, aunque se aproxima, no existe una relación entre los valores ausentes y el nivel de educación.

Supuesto No 2

Vamos a tomar otra columna para conocer si hay alguna otra relación con ella o descartarla. Ahora tomaremos la columna de `income_type`. Se piensa que el tipo de empleo puede ser otro factor por el cual un cliente no tendría experiencia laboral y tampoco reciba ingresos. Por ejemplo algunos que estén retirados o se encuentren desempleados, vamos a explorar un poco en ello.

Revisando distribución de clientes por ingreso

In [ ]:
```
dataclients['income_type'].isna().sum()
```

Out[ ]:
```
0
```

In [ ]:
```
dataclients['income_type'].value_counts(dropna=False)
```

Out[ ]:
```
employee                       11119
business                        5085
retiree                         3856
civil servant                   1459
unemployed                         2
entrepreneur                       2
student                            1
paternity / maternity leave        1
Name: income_type, dtype: int64
```

Analizando valores ausentes

Hagamos un filtro entre el tipo de empleo y los clientes cuyo estatus actual se encuentran retiradas

In [ ]:
```
dataclients[(dataclients['days_employed'].isna()) & (dataclients['inco
```

Out[ ]:

	children	days_employed	dob_years	education	education_id	family_st
12	0	NaN	65	secondary education	1	partne
29	0	NaN	63	secondary education	1	unma
55	0	NaN	54	secondary education	1	partne
67	0	NaN	52	bachelor's degree	0	ma
145	0	NaN	62	secondary education	1	ma
...	...	...	...	...	...	
21311	0	NaN	49	secondary education	1	ma
21321	0	NaN	56	secondary education	1	ma
21414	0	NaN	65	secondary education	1	ma
21415	0	NaN	54	secondary education	1	ma
21423	0	NaN	63	secondary education	1	ma

413 rows × 12 columns

> Con estos datos nos damos cuenta de que solo 413 personas están retiradas y esto justificaría la falta de días de experiencia laboral; sin embargo en el resto de valores los clientes cuentan con un empleo en la actualidad, lo cual no justifica la falta de estos valores, vamos a ver si existe algún otro valor de desempleo para conocer esta relación, para ello, tomaremos la columna `employee` en esta ocasión.

In [ ]:

```
dataclients[(dataclients['days_employed'].isna()) & (dataclients['inco
```

Out[ ]:

	children	days_employed	dob_years	education	education_id	family_st
82	2	NaN	50	bachelor's degree	0	ma
83	0	NaN	52	secondary education	1	ma
90	2	NaN	35	bachelor's degree	0	ma
96	0	NaN	44	secondary education	1	ma
97	0	NaN	47	bachelor's degree	0	ma
...	...	...	...	...	...	
21432	1	NaN	38	some college	2	unma
21463	1	NaN	35	bachelor's degree	0	partne
21495	1	NaN	50	secondary education	1	partne
21502	1	NaN	42	secondary education	1	ma
21510	2	NaN	28	secondary education	1	ma

1105 rows × 12 columns

> Podemos ver que existen solo 1,105 datos de los 2,174 ausentes. Podemos concluir entonces que es posible que haya ocurrido un error al ingresar los datos o que los clientes no hayan querido responder a estos valores por cuestiones de tipo emocional ya sea miedo o timidez pero no podemos saberlo porque no existe algún otro patrón en específico que nos lo indíque. En general no existen patrones que nos puedan indicar la aparición de estos valores ausentes.

Comparación

A continuación vamos a crear una visión general de dos dataframes, una con los datos nulos y otra sin ellos para poder visualizar sus distribuciones.

In [ ]:
```
#DataFrame con valores ausentes
dataclients.describe()
```

Out[ ]:

	children	days_employed	dob_years	education_id	family_statu
count	21525.000000	19351.000000	21525.000000	21525.000000	21525.00
mean	0.538908	63046.497661	43.293380	0.817236	0.97
std	1.381587	140827.311974	12.574584	0.548138	1.42
min	-1.000000	-18388.949901	0.000000	0.000000	0.00
25%	0.000000	-2747.423625	33.000000	1.000000	0.00
50%	0.000000	-1203.369529	42.000000	1.000000	0.00
75%	1.000000	-291.095954	53.000000	1.000000	1.00
max	20.000000	401755.400475	75.000000	4.000000	4.00

In [ ]:
```
#DataFrame sin valores ausentes
dataclients.dropna().describe()
```

Out[ ]:

	children	days_employed	dob_years	education_id	family_statu
count	19351.000000	19351.000000	19351.000000	19351.000000	19351.000
mean	0.537388	63046.497661	43.255336	0.819079	0.972
std	1.371408	140827.311974	12.579170	0.550104	1.420
min	-1.000000	-18388.949901	0.000000	0.000000	0.000
25%	0.000000	-2747.423625	33.000000	1.000000	0.000
50%	0.000000	-1203.369529	42.000000	1.000000	0.000
75%	1.000000	-291.095954	53.000000	1.000000	1.000
max	20.000000	401755.400475	75.000000	4.000000	4.000

Conclusión

Podemos observar con certeza que en ambos dataframes son prácticamente iguales y no existe una distribución especial para alguna de nuestras variables lo cual nos lleva a la conclusión de que nuestros datos nulos se generaron al azar.

Etapa 2: Transformación de datos

Durante esta etapa vamos a repasar cada columna para ver qué problemas podemos encontrar en ellas, comenzaremos con la eliminación de valores duplicados y continuaremos con la corrección de la información educativa en caso de ser necesario.

Revisando Columna: Education

In [ ]:
```
dataclients['education'].unique()
```

Out[ ]:
```
array(["bachelor's degree", 'secondary education', 'some college',
       'primary education', 'graduate degree'], dtype=object)
```

> Los valores se arreglaron previamente por lo que no será necesario tratar esta columna.

Columna: Children

Distribución en la columna

In [ ]:
```
dataclients['children'].value_counts(dropna=False)
```

Out[ ]:
```
 0     14149
 1      4818
 2      2055
 3       330
 20       76
-1        47
 4        41
 5         9
Name: children, dtype: int64
```

> Notamos que en la columna de **Children** hay valores negativos, es probable que se trate de un error de entrada en los datos o quizás al importar el dataframe, no puede existir un valor de **-1 hijos**. Calcularemos primero el porcentaje que representan estos datos respecto al resto del conjunto y después cambiaremos su **valor en positivo**. También tenemos un registro de 76 clientes con 20 hijos, algo que es muy improbable, lo más seguro es que hayan anotado un cero de más y se trate de 2 hijos, además se trata de un valor muy bajo que no impactará en nuestro análisis si lo agregamos a la columna con 2 hijos.

Distribución de negativos

In [ ]:
```
total_data = 21525
negative_children = 47

negative_children_percentage = negative_children/total_data
f'El porcentaje de valores negativos en la columna children es: {(nega
```

Out[ ]:
```
'El porcentaje de valores negativos en la columna children es: 0%'
```

Con esta operación podemos ver que el número de datos negativos no es relevante para nuestro análisis y podrían quedarse así; sin embargo esto no es correcto y cambiaremos los valores negativos a positivos.

In [ ]:
```
# Imprimiendo valores
dataclients['children'].unique()
```

Out[ ]:
```
array([ 1,  0,  3,  2, -1,  4, 20,  5])
```

In [ ]:
```
# Corrección de valores / verificando cambios
dataclients['children'] = dataclients['children'].abs()
dataclients['children'] = dataclients['children'].replace(20,2)
dataclients['children'].unique()
```

Out[ ]:
```
array([1, 0, 3, 2, 4, 5])
```

Columna: Days Employed

Revisando distribución de valores

In [ ]:
```
dataclients['days_employed'].value_counts()
```

Out[ ]:
```
-8437.673028      1
-3507.818775      1
 354500.415854    1
-769.717438       1
-3963.590317      1
                 ..
-1099.957609      1
-209.984794       1
 398099.392433    1
-1271.038880      1
-1984.507589      1
Name: days_employed, Length: 19351, dtype: int64
```

In [ ]:
```
dataclients['days_employed'].unique()
```

Out[ ]:
```
array([-8437.67302776, -4024.80375385, -5623.42261023, ...,
       -2113.3468877 , -3112.4817052 , -1984.50758853])
```

En esta columna encontramos los siguientes problemas:

1. Valores NaN
2. Valores negativos.
3. Valores de coma flotante.

Estos problemas pueden deberse a errores en la manipulación de la información o quizás al factor humano al ingresar los datos al dataframe. Sea cual sea la causa debemos solucionarlo.

Vamos abordar estos problemas de la siguiente manera:

1. Los valores ausentes los colocaremos en cero.
2. Los valores negativos los cambiaremos a positivos.
3. Los decimales los pasaremos a enteros pues es difícil interpretar 35.2 días.

In [ ]:
```
dataclients['days_employed'] = dataclients['days_employed'].fillna(0)
dataclients['days_employed'] = dataclients['days_employed'].abs()
dataclients['days_employed'] = dataclients['days_employed'].astype(int
```

Comprobando correcciones

In [ ]:
```
dataclients['days_employed'].value_counts()
```

Out[ ]:
```
0         2174
133         16
327         16
438         15
223         14
          ...
8200         1
9090         1
360849       1
2101         1
343937       1
Name: days_employed, Length: 9087, dtype: int64
```

Al aplicar 'value_counts' podemos ver que existen valores muy grandes, quizás estén expresados en otras unidades, vamos a intentar convertirlos en horas.

In [ ]:
```
dataclients['days_employed'].max()
```

Out[ ]:
```
401755
```

In [ ]:
```
dataclients['days_employed'] = round(dataclients['days_employed'] / 24
dataclients['days_employed'] = dataclients['days_employed'].astype(int
dataclients['days_employed'].max()
```

Out[ ]:
```
16740
```

Ahora el valor máximo es de 16740 días que equivalen a 45 años, ahora nuestros datos tienen más sentido.

Columna: Dob Years

Ahora echemos un vistazo a la edad de clientes para ver si hay algún problema allí.

In [ ]:
```
dataclients['dob_years'].unique()
```

Out[ ]:
```
array([42, 36, 33, 32, 53, 27, 43, 50, 35, 41, 40, 65, 54, 56, 26, 4
8, 24,
       21, 57, 67, 28, 63, 62, 47, 34, 68, 25, 31, 30, 20, 49, 37, 4
5, 61,
       64, 44, 52, 46, 23, 38, 39, 51,  0, 59, 29, 60, 55, 58, 71, 2
2, 73,
       66, 69, 19, 72, 70, 74, 75])
```

Análisis de distribución

In [ ]:
```
dataclients['dob_years'].value_counts()
```

Out[ ]:
```
35    617
40    609
41    607
34    603
38    598
42    597
33    581
39    573
31    560
36    555
44    547
29    545
30    540
48    538
37    537
50    514
43    513
32    510
49    508
28    503
45    497
27    493
56    487
52    484
47    480
54    479
46    475
58    461
57    460
```

```
53    459
51    448
59    444
55    443
26    408
60    377
25    357
61    355
62    352
63    269
64    265
24    264
23    254
65    194
22    183
66    183
67    167
21    111
0     101
68     99
69     85
70     65
71     58
20     51
72     33
19     14
73      8
74      6
75      1
Name: dob_years, dtype: int64
```

Existe un total de 101 registros en cero (0), esto no puede ser posible debido a que un cliente no podría tener cero años, de ser ese el caso estaríamos hablando de un recién nacido. Esta cantidad corresponde al 0.4% del total de los datos lo cuál no impacta en nuestro análisis.

Vamos a reemplazar estos valores por la media.

In [ ]:
```
m = dataclients['dob_years'].mean()
print('El valor promedio de la columna dob years es:', m)
```

```
El valor promedio de la columna dob years es: 43.29337979094077
```

In [ ]:
```
dataclients['dob_years'].replace(0,43, inplace=True)
dataclients['dob_years'].value_counts()
```

Out[ ]:
```
35    617
43    614
40    609
41    607
34    603
38    598
```

```
42    597
33    581
39    573
31    560
36    555
44    547
29    545
30    540
48    538
37    537
50    514
32    510
49    508
28    503
45    497
27    493
56    487
52    484
47    480
54    479
46    475
58    461
57    460
53    459
51    448
59    444
55    443
26    408
60    377
25    357
61    355
62    352
63    269
64    265
24    264
23    254
65    194
66    183
22    183
67    167
21    111
68     99
69     85
70     65
71     58
20     51
72     33
19     14
73      8
74      6
75      1
Name: dob_years, dtype: int64
```

Columna: Family_Status

In [ ]:
```
print('Valores únicos:', dataclients['family_status'].unique())
dataclients['family_status'].value_counts(dropna=False)
```

```
Valores únicos: ['married' 'civil partnership' 'widow / widower' 'divor
ced' 'unmarried']
```

Out[ ]:
```
married              12380
civil partnership     4177
unmarried             2813
divorced              1195
widow / widower        960
Name: family_status, dtype: int64
```

No se aprecian valores extraños relevantes para nuestro análisis.

Columna: Gender

Vamos a revisar ahora la columna 'gender' para observar si hay problemas en ella.

In [ ]:
```
print('valores únicos:', dataclients['gender'].unique())
dataclients['gender'].value_counts()
```

```
valores únicos: ['F' 'M' 'XNA']
```

Out[ ]:
```
F      14236
M       7288
XNA        1
Name: gender, dtype: int64
```

Encontramos un valor extraño llamado 'XNA', como no se trata de un valor representativo podemos integrarlo a cualquiera de los dos géneros sin que genere un impacto en nuestro análisis, en mi caso lo integraré al género F debido a la frecuencia que tiene la columna.

In [ ]:
```
dataclients['gender'] = dataclients['gender'].replace('XNA','F')
dataclients['gender'].value_counts()
```

Out[ ]:
```
F    14237
M     7288
Name: gender, dtype: int64
```

Columna: Income Type

Vamos a revisar la columna 'income_type'.

In [ ]:
```
print('valores únicos:', dataclients['income_type'].unique())
dataclients['income_type'].value_counts(dropna=False)
```

```
valores únicos: ['employee' 'retiree' 'business' 'civil servant' 'unemp
loyed'
 'entrepreneur' 'student' 'paternity / maternity leave']
```

Out[ ]:
```
employee                       11119
business                        5085
retiree                         3856
civil servant                   1459
unemployed                         2
entrepreneur                       2
student                            1
paternity / maternity leave        1
Name: income_type, dtype: int64
```

No se aprecian valores extraños o relevantes para nuestro análisis.

DataFrame: Datos Duplicados

Revisemos si existen datos duplicados.

In [ ]:
```
print(
    'Total de datos duplicados:',
    dataclients.duplicated().sum())
```

```
Total de datos duplicados: 71
```

In [ ]:
```
# Abordando duplicados
dataclients = dataclients.drop_duplicates().reset_index(drop=True)
dataclients.duplicated().sum()
```

Out[ ]:
```
0
```

Comprobando tamaño del dataframe después de manipulaciones

In [ ]:
```
dataclients.shape
```

Out[ ]:
```
(21454, 12)
```

Conclusión

Después de limpiar algunas columnas podemos ver un ajuste en el número de valores, aún nos falta por revisar los valores ausentes de la columna 'total_income' pero es algo que resolveremos en la siguiente etapa.

Etapa 3: Trabajar con valores ausentes

Vamos a trabajar con algunos valores ausentes que encontramos en la columna **total_income**, pero primero vamos a revisar los valores de **índice y valor** en education y family status para visualizar un poco la correlación que existe entre

estas variables

In [ ]:
```
dataclients[['education_id', 'education']].value_counts()
```

Out[ ]:
```
education_id  education
1             secondary education    15172
0             bachelor's degree       5250
2             some college             744
3             primary education        282
4             graduate degree            6
dtype: int64
```

In [ ]:
```
dataclients[['family_status_id', 'family_status']].value_counts()
```

Out[ ]:
```
family_status_id  family_status
0                 married              12339
1                 civil partnership     4151
4                 unmarried             2810
3                 divorced              1195
2                 widow / widower        959
dtype: int64
```

In [ ]:
```
dataclients['dob_years'].value_counts().sort_values()
```

Out[ ]:
```
75      1
74      6
73      8
19     14
72     33
20     51
71     56
70     65
69     85
68     99
21    111
67    167
66    182
22    183
65    193
23    252
64    260
24    264
63    269
62    348
61    354
25    357
60    374
26    408
55    443
59    443
51    446
58    454
```

```
57    456
53    459
46    472
54    476
47    477
56    483
52    484
27    493
45    496
28    503
49    508
32    509
50    513
48    536
37    536
30    537
29    544
44    545
36    554
31    559
39    572
33    581
42    596
38    597
34    601
41    605
40    607
43    613
35    616
Name: dob_years, dtype: int64
```

Restaurar valores ausentes en `total_income`

Para restaurar valores ausentes en la columna vamos a crear una columna nueva de categorías por edad dentro del dataframe, la utilizaré más adelante como referencia para tratar los valores ausentes.

In [ ]:
```
def age_range(row):
    if row['dob_years'] < 20:
        return '10-20'
    elif row['dob_years'] < 30:
        return '20-30'
    elif row['dob_years'] < 40:
        return '30-40'
    elif row['dob_years'] < 50:
        return '40-50'
    elif row['dob_years'] < 60:
        return '50-60'
    else:
        return '60+'
```

In [ ]:
```
dataclients.apply(age_range, axis=1)
```

Out[ ]:
```
0        40-50
1        30-40
2        30-40
3        30-40
4        50-60
         ...
21449    40-50
21450      60+
21451    30-40
21452    30-40
21453    40-50
Length: 21454, dtype: object
```

Realizando Clasificación por Edades

In [ ]:
```
dataclients['age_range'] = dataclients.apply(age_range, axis=1)
dataclients.head()
```

Out[ ]:

	children	days_employed	dob_years	education	education_id	family_status
0	1	352	42	bachelor's degree	0	married
1	1	168	36	secondary education	1	married
2	0	234	33	secondary education	1	married
3	3	172	32	secondary education	1	married
4	0	14178	53	secondary education	1	civil partnership

> Uno de los factores que nos puede ayudar a rellenar los valores ausentes en la columna de ingresos es el rango de edad y para ello utilizaremos nuestra columna con el rango de edades, para ello, crearemos una tabla que solo tenga datos sin valores ausentes, estos datos se utilizarán para restaurar los valores ausentes.

In [ ]:
```
data_ref = dataclients.dropna()
data_ref.head()
```

Out[ ]:

	children	days_employed	dob_years	education	education_id	family_status
0	1	352	42	bachelor's degree	0	married
1	1	168	36	secondary education	1	married
2	0	234	33	secondary education	1	married
3	3	172	32	secondary education	1	married
4	0	14178	53	secondary education	1	civil partnership

Examinando promedio y mediana

Vamos analizar los promedios de los ingresos en función de los factores que clasificamos.

In [ ]:
```
mean = data_ref.groupby('age_range')['total_income'].mean()
print('Promedio de ingresos por rango de edad:', mean)
```

```
Promedio de ingresos por rango de edad: age_range
10-20    16993.942462
20-30    25572.630177
30-40    28312.479963
40-50    28491.929026
50-60    25811.700327
60+      23021.639994
Name: total_income, dtype: float64
```

In [ ]:

In [ ]:
```
median = data_ref.groupby('age_range')['total_income'].median()
print('Mediana de ingresos por rango de edad:', median)
```

```
Mediana de ingresos por rango de edad: age_range
10-20    14934.9010
20-30    22799.2580
30-40    24667.5280
40-50    24755.6960
50-60    22203.0745
60+      19761.4250
Name: total_income, dtype: float64
```

In [ ]:

Vamos a utilizar el promedio para sustituir valores ausentes en el ingreso de los

clientes, para ello, vamos a desarrollar una función que nos ayude a sustituir los NaN por el promedio que corresponde a la clasificación de rangos por edad.

In [ ]:
```
def missing_replace(dataclients):
    mean = dataclients.groupby('age_range')['total_income'].mean()

    dataclients.loc[(dataclients['age_range'] == '10-20') & dataclient
    dataclients.loc[(dataclients['age_range'] == '20-30') & dataclient
    dataclients.loc[(dataclients['age_range'] == '30-40') & dataclient
    dataclients.loc[(dataclients['age_range'] == '40-50') & dataclient
    dataclients.loc[(dataclients['age_range'] == '50-60') & dataclient
    dataclients.loc[(dataclients['age_range'] == '60+') & dataclients[

    return dataclients
```

In [ ]:
```
# Comprobando función
missing_replace(dataclients)
```

Out[ ]:

	children	days_employed	dob_years	education	education_id	family_st
0	1	352	42	bachelor's degree	0	ma
1	1	168	36	secondary education	1	ma
2	0	234	33	secondary education	1	ma
3	3	172	32	secondary education	1	ma
4	0	14178	53	secondary education	1	partne
...	...	...	...	...	...	
21449	1	189	43	secondary education	1	partne
21450	0	14331	67	secondary education	1	ma
21451	1	88	38	secondary education	1	partne
21452	3	130	38	secondary education	1	ma
21453	2	83	40	secondary education	1	ma

21454 rows × 13 columns

Sustituyendo Valores Ausentes

In [ ]:

```
dataclients = missing_replace(dataclients)
ti_nan = dataclients['total_income'].isna().sum()
print('Valores ausentes en total income:', ti_nan)
dataclients.head()
```

```
Valores ausentes en total income: 0
```

Out[ ]:

	children	days_employed	dob_years	education	education_id	family_status
0	1	352	42	bachelor's degree	0	married
1	1	168	36	secondary education	1	married
2	0	234	33	secondary education	1	married
3	3	172	32	secondary education	1	married
4	0	14178	53	secondary education	1	civil partnership

Restaurar valores en `days_employed`

Ahora vamos a rellenar los valores ausentes en función del tipo de empleo pues tiene sentido que la experiencia laboral este en función de esta columna, para ello, buscaremos los datos de la media, la mediana y después, determinaremos cuales datos serán mejores para sustituirlos.

In [ ]:
```
# Contar Ceros
dataclients['days_employed'].value_counts()
```

Out[ ]:
```
0        2103
8         201
9         185
10        169
14        161
         ...
14090       1
16323       1
15420       1
15682       1
15217       1
Name: days_employed, Length: 2616, dtype: int64
```

In [ ]:
```
# Distribución de las medianas de `days_employed` en función de los pa
gender_median = dataclients.groupby('income_type')['days_employed'].me
gender_median
```

Out[ ]:
```
income_type
business                          55.0
civil servant                    100.0
employee                          57.0
entrepreneur                      11.0
paternity / maternity leave      137.0
retiree                        15032.0
student                           24.0
unemployed                     15267.5
Name: days_employed, dtype: float64
```

In [ ]:
```
# Distribución de las medias de `days_employed` en función de los pará
gender_mean = dataclients.groupby('income_type')['days_employed'].mean
gender_mean
```

Out[ ]:
```
income_type
business                          79.0
civil servant                    128.0
employee                          88.0
entrepreneur                      11.0
paternity / maternity leave      137.0
retiree                        13675.0
student                           24.0
unemployed                     15268.0
Name: days_employed, dtype: float64
```

Recordemos que nuestros datos nulos los transformamos en valores en ceros, por lo tanto, ahora vamos a sustituirlos por el valor de la mediana, para ello, vamos a desarrollar otra función que nos ayude a realizar esta sustitución.

In [ ]:
```
def replace_days_employed(dataclients):
    gender_median = dataclients.groupby('income_type')['days_employed'

    dataclients.loc[(dataclients['income_type'] == 'business' ) & (dat
    dataclients.loc[(dataclients['income_type'] == 'civil servant' ) &
    dataclients.loc[(dataclients['income_type'] == 'employee' ) & (dat
    dataclients.loc[(dataclients['income_type'] == 'paternity / matern
    dataclients.loc[(dataclients['income_type'] == 'retiree' ) & (data
    dataclients.loc[(dataclients['income_type'] == 'student' ) & (data
    dataclients.loc[(dataclients['income_type'] == 'unemployed' ) & (d

    return dataclients
```

In [ ]:
```
# Comprobando la función
replace_days_employed(dataclients)
```

Out[ ]:

	children	days_employed	dob_years	education	education_id	family_st
0	1	352.0	42	bachelor's degree	0	ma
1	1	168.0	36	secondary education	1	ma
2	0	234.0	33	secondary education	1	ma
3	3	172.0	32	secondary education	1	ma
4	0	14178.0	53	secondary education	1	partne
...	...	...	...	...	...	
21449	1	189.0	43	secondary education	1	partne
21450	0	14331.0	67	secondary education	1	ma
21451	1	88.0	38	secondary education	1	partne
21452	3	130.0	38	secondary education	1	ma
21453	2	83.0	40	secondary education	1	ma

21454 rows × 13 columns

Sustituyendo valores en days employed

In [ ]:

```
# Aplicar la función al income_type
dataclients = replace_days_employed(dataclients)
dataclients['days_employed'].value_counts()
```

Out[ ]:
```
57.0       1156
55.0        614
15032.0     388
100.0       220
8.0         201
           ...
16323.0       1
15420.0       1
15682.0       1
15672.0       1
15217.0       1
Name: days_employed, Length: 2616, dtype: int64
```

In [ ]:
```
# Conteo de NaN
dataclients['days_employed'].isna().sum()
```

Out[ ]:
```
0
```

Comprobando NaN en el Dataset

In [ ]:
```
dataclients.isna().sum()
```

Out[ ]:
```
children            0
days_employed       0
dob_years           0
education           0
education_id        0
family_status       0
family_status_id    0
gender              0
income_type         0
debt                0
total_income        0
purpose             0
age_range           0
dtype: int64
```

Etapa 4 : Clasificación de datos

Para poder responder a las preguntas y probar las diferentes hipótesis vamos a trabajar en esta sección con datos clasificados. A continuación vamos a clasificar los datos en función de las siguientes columnas:

1. Cantidad de Hijos
2. Incumplimiento de Pagos
3. Estado Civil
4. Ingreso Mensual

Esto para comprobar o rechazar las hipótesis que se establecieron desde el

inicio.

In [ ]:
```
# Valores de los datos seleccionados para la clasificación
dataclients['children'].value_counts()
```

Out[ ]:
```
0    14091
1     4855
2     2128
3      330
4       41
5        9
Name: children, dtype: int64
```

En este punto vamos a clasificar la columna 'debt' creando una nueva columna llamada 'debt_name' en la cual, para fines prácticos, se colocarán las siguientes anotaciones: no deudor = 0 y deudor = 1, para lograrlo, desarrollaremos una función que nos permita clasificar los datos.

In [ ]:
```
# Contando valores
dataclients['debt'].value_counts()
```

Out[ ]:
```
0    19713
1     1741
Name: debt, dtype: int64
```

In [ ]:
```
#Creamos una función para nuestra nueva columna
def debt_name(row):
    if row['debt'] == 0:
        return 'no deudor'
    else:
        return 'deudor'
```

In [ ]:
```
# Comprobamos que funcione
dataclients.apply(debt_name, axis=1)
```

Out[ ]:
```
0        no deudor
1        no deudor
2        no deudor
3        no deudor
4        no deudor
           ...
21449    no deudor
21450    no deudor
21451       deudor
21452       deudor
21453    no deudor
Length: 21454, dtype: object
```

Creando clasificación en Debt

In [ ]:
```
dataclients['debt_name'] = dataclients.apply(debt_name, axis=1)
dataclients.head()
```

Out[ ]:

	children	days_employed	dob_years	education	education_id	family_status
0	1	352.0	42	bachelor's degree	0	married
1	1	168.0	36	secondary education	1	married
2	0	234.0	33	secondary education	1	married
3	3	172.0	32	secondary education	1	married
4	0	14178.0	53	secondary education	1	civil partnership

Revisamos el resto de variables que vamos a utilizar

In [ ]:
```
dataclients['family_status'].value_counts()
```

Out[ ]:
```
married              12339
civil partnership     4151
unmarried             2810
divorced              1195
widow / widower        959
Name: family_status, dtype: int64
```

In [ ]:
```
dataclients['total_income'].value_counts()
```

Out[ ]:
```
28312.479963    553
28491.929026    530
25811.700327    479
25572.630177    295
23021.639994    245
               ...
23686.835000      1
9606.294000       1
28156.762000      1
24931.195000      1
13127.587000      1
Name: total_income, Length: 19354, dtype: int64
```

Vamos a comprobar los valores únicos

In [ ]:
```
# Comprobar los valores únicos
dataclients['children'].unique()
```

Out[ ]:
```
array([1, 0, 3, 2, 4, 5])
```

In [ ]:
```
dataclients['debt'].unique()
```

Out[ ]:
```
array([0, 1])
```

In [ ]:
```
dataclients['family_status'].unique()
```

Out[ ]:
```
array(['married', 'civil partnership', 'widow / widower', 'divorced',
       'unmarried'], dtype=object)
```

In [ ]:
```
dataclients['total_income'].unique()
```

Out[ ]:
```
array([40620.102, 17932.802, 23341.752, ..., 14347.61 , 39054.888,
       13127.587])
```

> Bloque con sangría

La siguiente clasificación será si tienen hijos y el incumplimiento de un pago, para ello, vamos a crear una nueva columna con esta clasificación de hijos que usaremos más adelante para conocer estos datos.

In [ ]:
```
# Función para clasificar los datos
children_ref = dataclients[['children', 'debt']]

def children_range(row):
    if row['children'] == 0:
        return 'sin hijos'
    else:
        return 'con hijos'
```

In [ ]:
```
# Creando columna con las categorías y cuenta los valores en ellas
dataclients['children_range'] = dataclients.apply(children_range, axis
```

In [ ]:
```
# Revisando todos los datos numéricos en la columna seleccionada para
dataclients['children_range'].value_counts()
```

Out[ ]:
```
sin hijos    14091
con hijos     7363
Name: children_range, dtype: int64
```

In [ ]:
```
# Estadísticas resumidas para la columna
dataclients['children_range'].describe()
```

Out[ ]:
```
count         21454
unique            2
top       sin hijos
freq          14091
Name: children_range, dtype: object
```

Clasificación del ingreso mensual

In [ ]:
```
# Estadísticas resumidas
dataclients['total_income'].describe()
```

Out[ ]:
```
count     21454.000000
mean      26787.022706
std       15657.948215
min        3306.762000
25%       17213.621250
50%       24565.763500
75%       31330.237250
max      362496.645000
Name: total_income, dtype: float64
```

Vamos a crear la función utilizando rangos en multiplos de 3, la razón de esto es porque tomamos como referencia el dataframe con la función describe y sus valores.

In [ ]:
```
# Función para clasificar en diferentes grupos numéricos basándose en
def income_range(row):
    if row['total_income'] < 3000:
        return '0 - 3000'
    elif row['total_income'] < 9000:
        return '3000 - 9000'
    elif row['total_income'] < 15000:
        return '9000 - 15000'
    elif row['total_income'] < 21000:
        return '15000 - 21000'
    elif row['total_income'] < 27000:
        return '21000 - 27000'
    else:
        return '27000+'
```

Creando columna nueva con categorías de ingresos

In [ ]:
```
dataclients['income_range'] = dataclients.apply(income_range, axis=1)
```

In [ ]:
```
# Distribución de categorías
dataclients['income_range'].value_counts().sort_values()
```

Out[ ]:
```
3000 - 9000       636
9000 - 15000     3107
15000 - 21000    4343
21000 - 27000    4947
27000+           8421
Name: income_range, dtype: int64
```

Vamos a clasificar la columna de 'purpose'

In [ ]:
```
# Función para clasificar los datos
def change_purpose(row):
    purpose_category='unknow'

    if 'wedding' in row['purpose']:
        purpose_category='wedding'


    elif ('real' in row['purpose']) or ('state' in row['purpose']) or
        purpose_category='real estate'


    elif 'car' in row['purpose']:
        purpose_category='car'

    elif ('education' in row['purpose']) or ('educated' in row['purpos
        purpose_category='education'

    return purpose_category
```

In [ ]:
```
# Comprobamos la función
dataclients.apply(change_purpose, axis=1).value_counts()
```

Out[ ]:
```
real estate    10811
car             4306
education       4013
wedding         2324
dtype: int64
```

Creando nueva clasificación y verificamos

In [ ]:
```
dataclients['purpose_classified'] = dataclients.apply(change_purpose,
dataclients.head()
```

Out[ ]:

	children	days_employed	dob_years	education	education_id	family_status
0	1	352.0	42	bachelor's degree	0	married
1	1	168.0	36	secondary education	1	married
2	0	234.0	33	secondary education	1	married
3	3	172.0	32	secondary education	1	married
4	0	14178.0	53	secondary education	1	civil partnership

Etapa 5: Comprobación de las hipótesis

Una vez que tenemos todo nuestro set de datos clasificado ahora vamos a comprobar nuestras hipótesis.

HIPÓTESIS No1 : ¿EXISTE UNA CORRELACIÓN ENTRE TENER HIJOS Y PAGAR A TIEMPO?

In [ ]:
```
dataclients.groupby(['children', 'debt_name'])['debt'].value_counts()
```

Out[ ]:
```
children  debt_name  debt
0         deudor     1        1063
          no deudor  0       13028
1         deudor     1         445
          no deudor  0        4410
2         deudor     1         202
          no deudor  0        1926
3         deudor     1          27
          no deudor  0         303
4         deudor     1           4
          no deudor  0          37
5         no deudor  0           9
Name: debt, dtype: int64
```

In [ ]:
```
# Contamos el total de hijos
dataclients['children'].value_counts()
```

Out[ ]:
```
0    14091
1     4855
2     2128
3      330
4       41
5        9
Name: children, dtype: int64
```

In [ ]:
```
# Calculando la tasa de incumplimiento en función del número de hijos
tasa_childrens = { 'children' :[0,1,2,3,4,5] ,
         'deudor'   :[(1063/14091)*100, (445/4855)*100, (202/2128)*100
         'no deudor':[(13028/14091)*100, (4410/4855)*100, (1926/2128)*
                 }
```

In [ ]:
```
pd.DataFrame(tasa_childrens)
```

Out[ ]:

	children	deudor	no deudor
0	0	7.543822	92.456178
1	1	9.165808	90.834192
2	2	9.492481	90.507519
3	3	8.181818	91.818182
4	4	9.756098	90.243902
5	5	0.000000	100.000000

Conclusión

Gracias a nuestra investigación podemos ver el **9.7%** de los clientes con más deuda son aquellos que tienen 4 hijos respecto de quienes no tienen hijos que representan solo el **7.5%** de incumplimiento y al mismo tiempo tienen mayor tasa de cumplimiento.

Podríamos hablar de los clientes que tienen **5 hijos** pero solo se trata de 9 clientes con estas características por lo que no es una cantidad considerable dentro de nuestro dataframe.

Hay que poner más énfasis en aquellos que tienen entre **1 y 2 hijos** pues representan una gran cantidad de clientes y las tasas de incumplimiento también son altas llegando al **9.5%** y **9.2%** respectivamente.

HIPÓTESIS NO2 : ¿EXISTE UNA CORRELACIÓN ENTRE LA SITUACIÓN FAMILIAR Y EL PAGO A TIEMPO?

In [ ]:

```
# Comprobando los datos del estado familiar y los pagos a tiempo
# Total de deudores por familia y la suma de deudores en c/u
debt_group = dataclients.groupby('family_status').agg({'debt':['size',
debt_group
```

Out[ ]:

	debt	
	size	sum
family_status		
civil partnership	4151	388
divorced	1195	85
married	12339	931
unmarried	2810	274
widow / widower	959	63

In [ ]:

```
# Ahora queremos obtener la tasa de los pagos a tiempo (no deudores)
# Vamos a restar el grupo de deudores del total para obtener a los No
# Luego vamos a dividir y sacar el porcentaje para conocer la tasa de

cumplimiento = debt_group['debt']['size'] - debt_group['debt']['sum']
tasa_pagos_puntuales = cumplimiento / debt_group['debt']['size']
print('Las tasas de pagos puntuales por estado civil')
print('.............................')

tasa_pagos_puntuales*100
```

```
Las tasas de pagos puntuales por estado civil
.............................
```

Out[ ]:

```
family_status
civil partnership    90.652855
divorced             92.887029
married              92.454818
unmarried            90.249110
widow / widower      93.430657
dtype: float64
```

In [ ]:

```
# Calcular la tasa de incumplimiento basada en el estado familiar
# Ya teniamos el total y la suma de deudores previamente
# Hacemos la operación correspondiente para obtener la tasa de incumpl

tasa_incumplimiento = debt_group['debt']['sum'] / debt_group['debt']['
print('Las Tasas de incumplimiento por estado civil')
print('.............................')

tasa_incumplimiento * 100
```

```
Las Tasas de incumplimiento por estado civil
.............................
```

Out[ ]:
```
family_status
civil partnership    9.347145
divorced             7.112971
married              7.545182
unmarried            9.750890
widow / widower      6.569343
dtype: float64
```

Conclusión

En nuestra tabla de **incumplimiento** podemos ver que los **viudos** representan la menor tasa de incumplimiento respecto a los que viven en **unión libre** que representan hasta el **9.3%**. Por otra parte los **casados**, quienes representan la mayor cantidad de nuestro dataframe, representan el **7.5%** de incumplimiento.

HIPÓTESIS: ¿EXISTE UNA CORRELACIÓN ENTRE EL NIVEL DE INGRESOS Y EL PAGO A TIEMPO?

In [ ]:
```
# ComprOBANDO los datos del nivel de ingresos y los pagos a tiempo
income_ref = dataclients.groupby(['income_range','debt_name'])['debt']
income_ref
```

Out[ ]:
```
income_range   debt_name  debt
15000 - 21000  deudor     1        358
               no deudor  0       3985
21000 - 27000  deudor     1        438
               no deudor  0       4509
27000+         deudor     1        647
               no deudor  0       7774
3000 - 9000    deudor     1         41
               no deudor  0        595
9000 - 15000   deudor     1        257
               no deudor  0       2850
Name: debt, dtype: int64
```

In [ ]:
```
# Contamos el total de ingresos
income_total = dataclients.groupby('income_range').size()
income_total
```

Out[ ]:
```
income_range
15000 - 21000    4343
21000 - 27000    4947
27000+           8421
3000 - 9000       636
9000 - 15000     3107
dtype: int64
```

In [ ]:
```
# Distribución
income_conversion = (income_ref / income_total)*100
income_conversion
```

Out[ ]:
```
income_range   debt_name  debt
15000 - 21000  deudor     1        8.243150
               no deudor  0       91.756850
21000 - 27000  deudor     1        8.853851
               no deudor  0       91.146149
27000+         deudor     1        7.683173
               no deudor  0       92.316827
3000 - 9000    deudor     1        6.446541
               no deudor  0       93.553459
9000 - 15000   deudor     1        8.271645
               no deudor  0       91.728355
dtype: float64
```

Conclusión

La mayor **tasa de cumplimiento** la tienen aquellos clientes cuyos ingresos mensuales rondan los **3000 a 9000**. Por otra parte los clientes con más **incumplimiento** son aquellos que sus ingresos mensuales rondan entre los **21000 y 27000**. Es muy probable que esto se deba a que realmente son pocos los clientes con el nivel de ingreso de 3 a 9 mil mientras que la cantidad de clientes va a más del doble en aquellos que ganan entre 21 mil y los 27 mil.

¿Cómo afecta el propósito del crédito a la tasa de incumplimiento?

In [ ]:
```
# Consultando los porcentajes de tasa de incumplimiento para cada prop
debt_grouped = dataclients.groupby(['purpose_classified', 'debt_name']
```

In [ ]:
```
# Contamos el total de la columna 'purpose'
purpose = dataclients.groupby('purpose_classified').size()
```

In [ ]:
```
# Calculamos la distribución
purpose_conversion = (debt_grouped / purpose)*100
purpose_conversion
```

Out[ ]:
```
purpose_classified  debt_name  debt
car                 deudor     1        9.359034
                    no deudor  0       90.640966
education           deudor     1        9.220035
                    no deudor  0       90.779965
real estate         deudor     1        7.233373
                    no deudor  0       92.766627
wedding             deudor     1        8.003442
                    no deudor  0       91.996558
dtype: float64
```

Conclusión

En lo que respecta al propósito del crédito podemos ver que la tasa de deudores más alta, del 9%, pertenece a los clientes que han buscado invertir en un auto, en

general, podemos ver que el propósito de un crédito no afecta en el cumplimiento o incumplimiento de este.

Conclusión general

Hemos pasado un buen rato analizando datos e información clave y hemos encontrado diversas problematicas en todo el ejercicio como son:

1. Hemos resuelto problemas de valores ausentes en las columnas de la experiencia laboral y el ingreso mensual.
2. Hemos tenido que resolver problemas adicionales repasando cada columna.
3. Hemos hecho un análsis previo para encontrar algún patrón o factor común que tuviera relación con los valores ausentes de las columnas de experiencia laboral e ingreso mensual.
4. Abordamos el tema de valores ausentes en la columna de ingreso mensual creando clasificaciones por rango de edades y calculando la media y la mediana determinando cuál era la más apropiada forma de aplicar al conjunto de datos.
5. Abordamos el tema de los duplicados dentro de nuestro dataset.
6. Encontramos valores negativos y algunos valores improbables en las columnas de número de hijos, días de experiencia laboral y en ingreso mensual. Estos valores solo se restauraron y se ordenaron.
7. Se crearon nuevas clasificaciones de rango de edad, rango de ingreso mensual, rango de hijos y nombres para las columnas de deudores con el objetivo de analizar, responder y comprobar nuestras hiótesis.

Hipótesis

La primera hipótesis sobre la relación entre el número de hijos y el pago de un préstamo es aceptada. La segunda hipótesis sobre la relación entre el estado civil y el pago de un préstamo es aceptada. La tercera hipótesis sobre el nivel de ingreso y el pago de una deuda es aceptada. La cuarta hipótesis sobre cómo afectan las razones para pedir un préstamo y el incumplimiento aceptada pues la relación que existe es baja.

Conclusión

Después de este análisis determinamos que los aspectos más relevantes para solicitar un crédito son el número de hijos, el estado civil y el nivel de ingreso mensual. Mientras que las razones por las cuales éste se solicita no tienen mucha influencia en el cumplimiento del crédito.